The Air Force estimated that an additional $1.1 billion USD would be required to deliver just a quarter of the original scope, and that the solution could not be rolled out until 2020, three years after the legislative deadline of 2017.

This happens a lot in the software industry. According to a number of studies, around half of all software projects fail to deliver in some significant way. The 2011 edition of the Standish Group's annual *CHAOS Report* found that 42% of projects were delivered late, ran over budget, or failed to deliver all of the requested features,[2] and 21% of projects were cancelled entirely. Scott Ambler's annual survey on IT project success rates uses a more flexible definition of success, but still found a 30–50% failure rate, depending on the methodologies used.[3] This corresponds to billions of dollars in wasted effort, writing software that ultimately won't be used or that doesn't solve the business problem it was intended to solve.

What if it didn't have to be this way? What if we could write software in a way that would let us discover and focus our efforts on what really matters? What if we could objectively learn what features will really benefit the organization and the most cost-effective way to implement them? What if we could see beyond what the user asks for and build what the user actually needs?

There are organizations discovering how to do just that. Many teams are successfully collaborating to build and deliver more valuable, more effective, and more reliable software. And they're learning to do this faster and more efficiently. In this book, you'll see how—we'll explore a number of methods and techniques, grouped under the general heading of *Behavior-Driven Development* (BDD).

BDD helps teams focus their efforts on identifying, understanding, and building valuable features that matter to businesses, and it makes sure that these features are well designed and well implemented.

BDD practitioners use conversations around concrete examples of system behavior to help understand how features will provide value to the business. BDD encourages business analysts, software developers, and testers to collaborate more closely by enabling them to express requirements in a more testable way, in a form that both the development team and business stakeholders can easily understand. BDD tools can help turn these requirements into automated tests that help guide the developer, verify the feature, and document what the application does.

BDD isn't a software development methodology in its own right. It's not a replacement for Scrum, XP, Kanban, RUP, or whatever methodology you're currently using. As you'll see, BDD incorporates, builds on, and enhances ideas from many of these methodologies. And no matter what methodology you're using, there are ways that BDD can help make your life easier.

---

[2] Whether these figures reflect more on our ability to build and deliver software or on our ability to plan and estimate is a subject of some debate in the Agile development community—see Jim Highsmith's book *Agile Project Management: Creating Innovative Products*, second edition (Addison-Wesley Professional, 2009).

[3] Scott Ambler, Surveys Exploring the Current State of Information Technology Practices, http://www.ambysoft.com/surveys/.